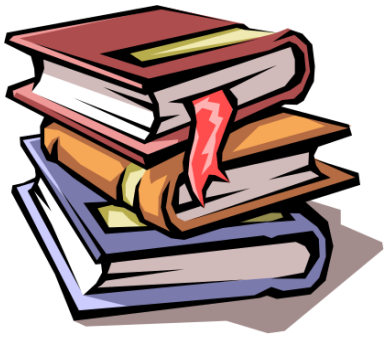
FPT Education

FPT UNIVERSITY

Lab Report

LAB-01 – KERNEL CONFIGURATION & EMBEDDED SYSTEM BOOT

Embedded Linux Systems – ARMv7 Cortex-A9 (QEMU vexpress-a9)



Students	Phạm Võ Hoàng Nhân Nguyễn Duy Khoa Ngô Từ Nhã Quyên Nguyễn Cao Kỳ
Class	IC2008
Lecturer	Phạm Thế Vinh
Date	27/09/2026

1. Group information and environment

Members	Phạm Võ Hoàng Nhân, Nguyễn Duy Khoa, Ngô Từ Nhã Quyên, Nguyễn Cao Kỳ
Class	IC2008
Date	27/09/2026
Host	Ubuntu 26.04, GCC 15.2, `arm-linux-gnueabihf` toolchain
Target	QEMU `vexpress-a9`, Cortex-A9 CPU (ARMv7-A)
Sources	Linux 5.15, U-Boot 2022.04, BusyBox 1.35.0

The lab handout was written for Ubuntu 20.04/22.04. Our host runs Ubuntu 26.04 with GCC 15.2, which is much newer than kernel 5.15 and BusyBox 1.35, so we had to patch two places in the build system before everything would build. Both problems are described in section 5.

2. Boot sequence diagram

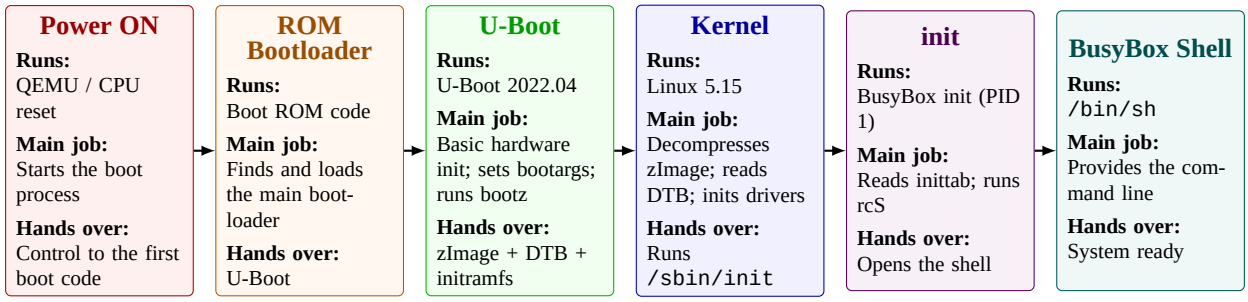


Figure 1: Boot sequence from power-on to the BusyBox shell

When QEMU powers on the machine, the Cortex-A9 is reset and jumps to the first boot code. On a real board this is the boot ROM; in QEMU this stage is emulated and hands over straight to U-Boot, the file we pass with `-kernel u-boot`. U-Boot brings up the minimum hardware (UART, memory), reads its environment variables and runs `bootcmd`. In this lab `bootcmd` is

```
bootz 0x60100000 0x64000000 0x60000000
      zImage     initramfs  DTB
```

which means U-Boot jumps into the zImage at `0x60100000` and tells the kernel that the initramfs is at `0x64000000` and the Device Tree is at `0x60000000`. The `bootargs` string (console, root, init, mem) is written by U-Boot into the DTB, where the kernel reads it as its command line.

The kernel decompresses itself, calls `start_kernel()`, reads the DTB to find out what hardware is present (PL011 UART, RAM, timers…), initialises the drivers, and unpacks the initramfs into RAM as the root filesystem. Its last step is to run `/sbin/init`; this is the switch from kernel space to user space. BusyBox init (PID 1) reads `/etc/inittab`, runs `/etc/init.d/rcS` to mount `/proc`, `/sys`, `/tmp` and set the hostname, and then opens `/bin/sh` on the console. From here the system is ready for commands.

3. Lab logs

Part A – Building the kernel

We used `vexpress_defconfig` rather than the `versatile_defconfig` named in the submission checklist, because the target of this lab is vexpress-a9 (Cortex-A9). Versatile PB is an

ARM926 (ARMv5) board and does not match the CPU we are using.

```
make vexpress_defconfig
  HOSTCC  scripts/basic/fixdep
  HOSTCC  scripts/kconfig/conf.o
  HOSTCC  scripts/kconfig/confdata.o
  HOSTCC  scripts/kconfig/expr.o
  LEX     scripts/kconfig/lexer.lex.c
  YACC    scripts/kconfig/parser.tab.[ch]
  HOSTCC  scripts/kconfig/lexer.lex.o
  HOSTCC  scripts/kconfig/menu.o
  HOSTCC  scripts/kconfig/parser.tab.o
  HOSTCC  scripts/kconfig/preprocess.o
  HOSTCC  scripts/kconfig/symbol.o
  HOSTCC  scripts/kconfig/util.o
  HOSTLD  scripts/kconfig/conf
#
# configuration written to .config
#
```

Figure 2: Output of `make vexpress_defconfig`

In menuconfig we confirmed `CONFIG_ARCH_VEXPRESS=y`, enabled initramfs support (General setup → Initial RAM filesystem and RAM disk support), kept the PL011 UART driver so that the `ttyAMA0` console is available, and enabled SMP.

```
  SYNC    include/config/auto.conf
  CALL    scripts/atomic/check-atomics.sh
  CALL    scripts/checksyscalls.sh
  CHK     include/generated/compile.h
  Kernel: arch/arm/boot/Image is ready
  Kernel: arch/arm/boot/zImage is ready
```

Figure 3: End of the build log: `Image` and `zImage` are ready

```
dyk@mydklaptop:~/embedded_lab1/kernel/linux-5.15$ cd ~/embedded_lab1/kernel/linux-5.15
ls -lh arch/arm/boot/zImage
-rwxrwxr-x 1 dyk dyk 4.9M Sep 27 19:04 arch/arm/boot/zImage
```

Figure 4: zImage size after the build: 4.9 MB

The zImage is 4.9 MB, within the 5–8 MB range the handout gives as normal for vexpress_defconfig. The `SMP` string in `uname -a` in part C also confirms that the intended configuration was built.

Part B – U-Boot

```
⇒ printenv
arch=arm
baudrate=38400
board=vexpress
board_name=vexpress
boot_a_script=load ${devtype} ${devnum}:${distro_bootpart} ${scriptaddr} ${prefix}${script}; source ${scriptad
dr}
boot_efi_binary=load ${devtype} ${devnum}:${distro_bootpart} ${kernel_addr_r} efi/boot/bootarm.efi; if fdt add
r ${fdt_addr_r}; then bootefi ${kernel_addr_r} ${fdt_addr_r};else bootefi ${kernel_addr_r} ${fdtcontroladdr};f
i
boot_efi_bootmgr=if fdt addr ${fdt_addr_r}; then bootefi bootmgr ${fdt_addr_r};else bootefi bootmgr;fi
boot_extlinux=sysboot ${devtype} ${devnum}:${distro_bootpart} any ${scriptaddr} ${prefix}${boot_syslinux_conf}
boot_prefixes=/ /boot/
boot_script_dhcp=boot.scr.uimg
boot_scripts=boot.scr.uimg boot.scr
boot_syslinux_conf=extlinux/extlinux.conf
boot_targets=mmc1 mmc0 pxe dhcp
bootargs=console=tty0 console=ttyAMA0,38400n8
bootcmd=run distro_bootcmd; run bootflash
bootcmd_dhcp=devtype=dhcp; if dhcp ${scriptaddr} ${boot_script_dhcp}; then source ${scriptaddr}; fi;setenv efi
_fdtfile ${fdtfile}; if test -z "${fdtfile}" -a -n "${soc}"; then setenv efi_fdtfile ${soc}-${board}${boardver
}.dtb; fi; setenv efi_old_vci ${bootp_vci};setenv efi_old_arch ${bootp_arch};setenv bootp_vci PXEClient:Arch:0
0010:UNDI:003000;setenv bootp_arch 0xa;if dhcp ${kernel_addr_r}; then tftpboot ${fdt_addr_r} dtb/${efi_fdtfile
};if fdt addr ${fdt_addr_r}; then bootefi ${kernel_addr_r} ${fdt_addr_r}; else bootefi ${kernel_addr_r} ${fdtc
ontroladdr};fi;fi;setenv bootp_vci ${efi_old_vci};setenv bootp_arch ${efi_old_arch};setenv efi_fdtfile;setenv
efi_old_arch;setenv efi_old_vci;
bootcmd_mmc0=devnum=0; run mmc_boot
bootcmd_mmc1=devnum=1; run mmc_boot
bootcmd_pxe=dhcp; if pxe get; then pxe boot; fi
bootdelay=2
bootflash=run flashargs; cp ${ramdisk_addr} ${ramdisk_addr_r} ${maxramdisk}; bootm ${kernel_addr} ${ramdisk_ad
dr_r}
console=ttyAMA0,38400n8
cpu=armv7
distro_bootcmd=for target in ${boot_targets}; do run bootcmd_${target}; done
dram=1024M
efi_dtb_prefixes=/ /dtb/ /dtb/current/
ethaddr=52:54:00:12:34:56
fdt_addr_r=0x60000000
fdtcontroladdr=7ffe0430
fdtfile=vexpress-v2p-ca9.dtb
flashargs=setenv bootargs root=${root} console=${console} mem=${dram} mtdparts=${mtd} mmci.fmax=190000 devtmpf
s.mount=0  vmalloc=256M
kernel_addr_r=0x60100000
```

Figure 5: `printenv` in U-Boot (first part)

```
kernel_addr_r=0x60100000
load_efi_dtb=load ${devtype} ${devnum}:${distro_bootpart} ${fdt_addr_r} ${prefix}${efi_fdtfile}
loadaddr=0x90000000
mmc_boot=if mmc dev ${devnum}; then devtype=mmc; run scan_dev_for_boot_part; fi
mtd=armflash:1M@0x800000(uboot),7M@0x1000000(kernel),24M@0x2000000(initrd)
root=/dev/sda1 rw
scan_dev_for_boot=echo Scanning ${devtype} ${devnum}:${distro_bootpart}...; for prefix in ${boot_prefixes}; do
 run scan_dev_for_extlinux; run scan_dev_for_scripts; done;run scan_dev_for_efi;
scan_dev_for_boot_part=part list ${devtype} ${devnum} -bootable devplist; env exists devplist || setenv devpli
st 1; for distro_bootpart in ${devplist}; do if fstype ${devtype} ${devnum}:${distro_bootpart} bootfstype; the
n run scan_dev_for_boot; fi; done; setenv devplist
scan_dev_for_efi=setenv efi_fdtfile ${fdtfile}; if test -z "${fdtfile}" -a -n "${soc}"; then setenv efi_fdtfil
e ${soc}-${board}${boardver}.dtb; fi; for prefix in ${efi_dtb_prefixes}; do if test -e ${devtype} ${devnum}:${
distro_bootpart} ${prefix}${efi_fdtfile}; then run load_efi_dtb; fi;done;run boot_efi_bootmgr;if test -e ${dev
type} ${devnum}:${distro_bootpart} efi/boot/bootarm.efi; then echo Found EFI removable media binary efi/boot/b
ootarm.efi; run boot_efi_binary; echo EFI LOAD FAILED: continuing...; fi; setenv efi_fdtfile
scan_dev_for_extlinux=if test -e ${devtype} ${devnum}:${distro_bootpart} ${prefix}${boot_syslinux_conf}; then
echo Found ${prefix}${boot_syslinux_conf}; run boot_extlinux; echo SCRIPT FAILED: continuing...; fi
scan_dev_for_scripts=for script in ${boot_scripts}; do if test -e ${devtype} ${devnum}:${distro_bootpart} ${pr
efix}${script}; then echo Found U-Boot script ${prefix}${script}; run boot_a_script; echo SCRIPT FAILED: conti
nuing...; fi; done
stderr=serial
stdin=serial
stdout=serial
ubifs_boot=env exists bootubipart || env set bootubipart UBI; env exists bootubivol || env set bootubivol boot
; if ubi part ${bootubipart} && ubifsmount ubi${devnum}:${bootubivol}; then devtype=ubi; run scan_dev_for_boot
; fi
vendor=armltd

Environment size: 4238/262140 bytes
⇒
```

Figure 6: `printenv` (last part)

The default U-Boot environment for vexpress already uses addresses in the `0x6xxxxxx` RAM window: `kernel_addr_r=0x60100000`, `fdt_addr_r=0x60000000`, `fdtfile=vexpress-v2p-ca9.dtb`, `cpu=armv7`. The default `bootargs` only sets the console at 38400 baud, so we replaced it:

```
⇒ printenv bootargs
bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M
```

Figure 7: `setenv bootargs` and checking it with `printenv bootargs`

Booting before any root filesystem existed, the kernel completed its initialisation and then stopped at the root mount step:

```
VFS: Cannot open root device "ram0" or unknown-block(0,0): error -6
Please append a correct "root=" boot option; here are the available partitions:
1f00          131072 mtdblock0
 (driver?)
1f01           32768 mtdblock1
 (driver?)
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.15.0 #1
Hardware name: ARM-Versatile Express
[<801103a0>] (unwind_backtrace) from [<8010c2c4>] (show_stack+0x10/0x14)
[<8010c2c4>] (show_stack) from [<80867f00>] (dump_stack_lvl+0x40/0x4c)
[<80867f00>] (dump_stack_lvl) from [<80863f64>] (panic+0xfc/0x2e4)
[<80863f64>] (panic) from [<80b01714>] (mount_block_root+0x1bc/0x264)
[<80b01714>] (mount_block_root) from [<80b01b98>] (prepare_namespace+0x148/0x184)
[<80b01b98>] (prepare_namespace) from [<8086c6b8>] (kernel_init+0x14/0x13c)
[<8086c6b8>] (kernel_init) from [<80100130>] (ret_from_fork+0x14/0x24)
Exception stack(0x810a7fb0 to 0x810a7ff8)
7fa0:                                     00000000 00000000 00000000 00000000
7fc0: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
7fe0: 00000000 00000000 00000000 00000000 00000013 00000000
---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0) ]---
```

Figure 8: Kernel panic with no rootfs

This is the result the handout asks us to observe. `bootargs` tells the kernel to use `root=/dev/ram0`, but no initramfs had been loaded into RAM, so `/dev/ram0` is empty. The kernel lists the block devices it can see (`mtdblock0` and `mtdblock1` are QEMU's emulated flash), cannot mount any of them as root, and `mount_block_root` calls `panic()`. The backtrace shows the path `kernel_init` → `prepare_namespace` → `mount_block_root` → `panic`, i.e. the failure happens exactly at the "mount rootfs" step of the diagram in section 2. The kernel is still running as PID 1 (`swapper/0`) and never reaches user space.

Part C – Root filesystem and booting to a shell

```
dyk@mydklaptop:~/embedded_lab1/rootfs$ ls initramfs/
bin  dev  etc  init  lib  proc  sbin  sys  tmp  usr
```

Figure 9: Layout of the `initramfs/` directory

BusyBox was built statically (`CONFIG_STATIC=y`), so the rootfs needs no shared libraries. After packing it as `initramfs.cpio.gz`, we booted QEMU with `-kernel zImage -dtb vexpress-v2p-ca9.dtb -initrd initramfs.cpio.gz`.

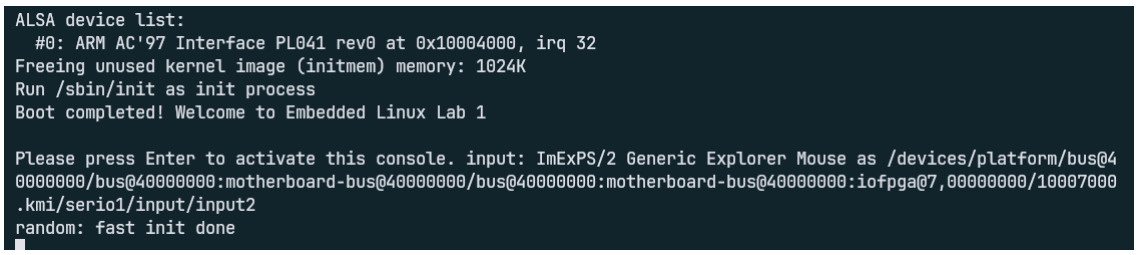


Figure 10: Successful boot: the kernel runs `/sbin/init` and rcS prints its message



Figure 11: `uname -a`: kernel 5.15.0, SMP, `armv7l` architecture

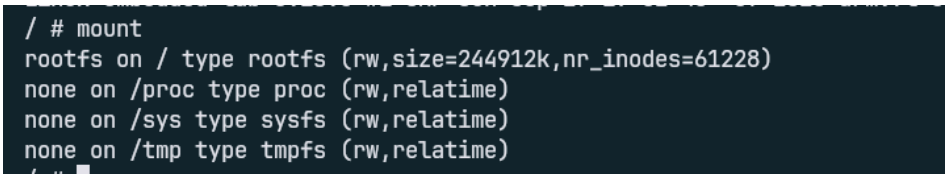


Figure 12: `mount`: rootfs, proc, sysfs, tmpfs

`mount` shows the three filesystems mounted by rcS (`proc`, `sysfs`, `tmpfs`) plus `rootfs`, which is the initramfs unpacked into RAM.

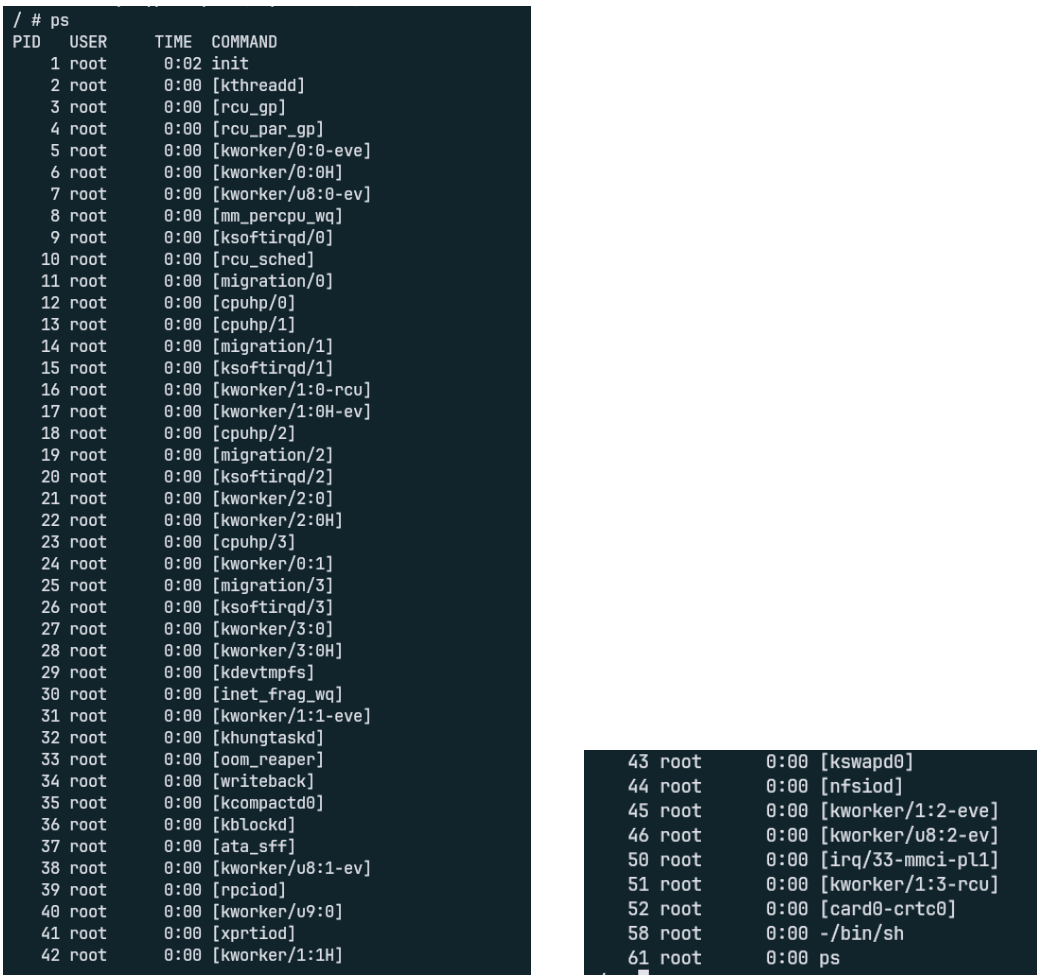


Figure 13: `ps`: PID 1 is `init`; almost everything else is a kernel thread

`ps` shows only three user-space processes: `init` (PID 1), `-/bin/sh` started by inittab, and the `ps` command itself. The entries in square brackets are kernel threads. Note the per-CPU threads such as `migration/N`, `ksoftirqd/N` and `cpuhp/N` with N from 0 to 3, which means the kernel is running on 4 CPUs (see question 5).

4. Answers to the analysis questions

Question 1. arm-linux-gnueabi vs arm-linux-gnueabihf

The two toolchains differ in their floating-point ABI. `gnueabi` uses the soft-float ABI: `float`/`double` arguments and return values are passed in integer registers (r0–r3), and by default it targets an older architecture (ARMv5TE) without assuming an FPU. `gnueabihf` uses the hard-float ABI: floating-point arguments go straight into VFP registers (s0–s15/d0–d7), and by default it targets ARMv7-A with VFPv3. The two ABIs cannot be mixed inside one program, because caller and callee must agree on which register holds each argument.

The Cortex-A9 has VFPv3 and NEON, so with `gnueabihf` floating-point operations run directly on the FPU, without copying values back and forth between integer and VFP registers.

For the kernel the story is a little different: Linux does not use floating point and is always built with `-msoft-float` (we saw exactly this flag in the build log, section 5). So if the kernel were built with `gnueabi`, `ARCH=arm` and an ARMv7 configuration, it would still build and boot. The problem is user space: BusyBox and other programs built with `gnueabi` would use soft-float,

which runs on the Cortex-A9 but does not use the FPU, and in a dynamic build they could not share libraries with hard-float programs. For the same reason the `KCFLAGS` with `-mfloat-abi=hard` suggested in the handout does not suit the kernel, so we removed it (`unset KCFLAGS`).

Question 2. Why the memory map starts at 0x60000000

Versatile PB places its SDRAM at the bottom of the address space, so RAM starts at `0x00000000`. Vexpress with the CoreTile Cortex-A9 daughterboard (V2P-CA9) uses a different map: the low region is reserved for the NOR boot flash and the motherboard peripherals (for example the PL011 UART at `0x10009000` and the PL041 audio controller at `0x10004000`, as seen in the boot log), while the daughterboard DRAM sits at `0x60000000`–`0x9FFFFFFF` (1 GB).

Every load address used in U-Boot therefore has to fall inside this window. We can see it directly in `printenv`: `kernel_addr_r=0x60100000`, `fdt_addr_r=0x60000000`. The command `bootz 0x60100000 0x64000000 0x60000000` puts the DTB at the start of RAM, the zImage 1 MB above it, and the initramfs at `0x64000000` (about 63 MB above the zImage, leaving room for the kernel to decompress itself without overwriting the initramfs). If we reused Versatile PB style addresses such as `0x00008000`, U-Boot would write into the flash/peripheral region instead of RAM and the kernel could not run. The kernel also needs to know where RAM starts; this comes from the `memory` node of the DTB, and because kernel 5.15 enables `ARM_PATCH_PHYS_VIRT` (the `p2v8` in the LAB-02 vermagic) the same zImage can run at any physical address.

Question 3. MIDR 0x410FC090

The Main ID Register is a read-only register that identifies the CPU core. Splitting the 32 bits of `0x410FC090`:

Bits	Field	Value	Meaning
31–24	Implementer	0x41	ARM Ltd. (letter ‘A’)
23–20	Variant	0x0	r0 (major revision)
19–16	Architecture	0xF	defined by the CPUID scheme, i.e. ARMv7
15–4	Part number	0xC09	Cortex-A9
3–0	Revision	0x0	p0 (minor revision)

So QEMU emulates a Cortex-A9 r0p0. At boot the kernel uses the part number to pick the matching `proc_info` for the CPU (cache, TLB, errata workarounds). The same value appears in `/proc/cpuinfo` as `CPU implementer: 0x41` and `CPU part: 0xc09`.

Question 4. Why BusyBox is built statically

The minimal initramfs in this lab contains only BusyBox, with no C library and no dynamic linker. A dynamic binary asks the loader `/lib/ld-linux-armhf.so.3` to load its `.so` libraries when it starts; if that file is missing, the kernel reports “not found” as soon as it tries to exec `/sbin/init`, and the system never reaches user space. A static build puts everything into one file, so the rootfs stays small and has no dependencies.

For a dynamic build, at least the following must be copied from the toolchain sysroot (`/usr/arm-linux-gnueabihf/lib/`) into `rootfs/lib/`: `ld-linux-armhf.so.3` (dynamic linker), `libc.so.6` (C library), `libm.so.6` (maths functions used by BusyBox), and, depending on the configuration, `libresolv.so.2`. The reliable way to know the list is complete is to run `arm-linux-gnueabihf-readelf -d busybox | grep NEEDED` and

copy exactly the files it lists.

Question 5. SMP on vexpress-a9

With SMP the kernel has extra work that a uniprocessor kernel does not. CPU0 boots first; the kernel then reads the number of CPUs from the DTB, enables the SCU (Snoop Control Unit) to keep the cores' caches coherent, and wakes each secondary CPU by writing the start address into the vexpress system flags register and sending an IPI through the GIC. Each secondary CPU runs `secondary_start_kernel()` and sets up its local timer and its own per-CPU data. The scheduler creates a runqueue for each CPU, along with per-CPU kernel threads (`migration`, `ksoftirqd`, `cpuhp`, `kworker`). Spinlocks now really lock, whereas on a uniprocessor kernel they amount to little more than disabling preemption.

The evidence in our lab is the `ps` output in part C: there are `migration/0..3`, `ksoftirqd/0..3` and `cpuhp/0..3`, so the kernel brought up 4 CPUs. With `-smp 1` these threads exist only as `/0`. In the boot log, the SMP build adds lines such as "CPU1: thread -1, cpu 1, socket 0" for each core and "SMP: Total of N processors activated".

The cost is a slightly larger image, because of the extra SMP code (IPIs, per-CPU data, real locking), and a slightly longer boot, because the kernel waits for each secondary CPU to come up. On a single-core embedded system, disabling SMP gives a smaller and faster image.

5. Remarks and difficulties

Problem 1: kernel 5.15 does not build with GCC 15

On the first build the assembler reported a long list of errors such as

```
Error: selected processor does not support `dsb st' in ARM mode
Error: selected processor does not support `dmb ish' in ARM mode
Error: selected processor does not support `ldrex' ...
```

`dsb`, `dmb`, `isb`, `ldrex` and `strex` are all ARMv7 instructions, so the errors meant the compiler believed the target was an older architecture. The full compile command in the log confirmed it: `-D__LINUX_ARM_ARCH__=7 -march=armv5t -Wa,-march=armv7-a -msoft-float`.

The cause is in kernel 5.15's `arch/arm/Makefile`:

```
arch-$(CONFIG_CPU_32v7) =-D__LINUX_ARM_ARCH__=7 \
  $(call cc-option,-march=armv7-a,\
         -march=armv5t -Wa$(comma)-march=armv7-a)
```

`cc-option` tests whether the compiler accepts `-march=armv7-a` and falls back to `-march=armv5t` if it does not. With a recent hard-float cross-GCC this test fails even though GCC does support ARMv7-A, so the kernel falls back to ARMv5T and the assembler rejects the ARMv7 instructions. We kept kernel 5.15 as the handout requires and changed only the ARMv7 line to

```
arch-$(CONFIG_CPU_32v7) =-D__LINUX_ARM_ARCH__=7 -march=armv7-a
```

(mainline kernels have since simplified this line in the same way). We left the ARMv6 and ARMv5 lines alone because the target does not use them. After rebuilding we got `Kernel: arch/arm/boot/zImage is ready`.

Problem 2: BusyBox menuconfig reports missing ncurses

BusyBox 1.35's `make menuconfig` reported "Unable to find the ncurses libraries" even though `libncurses-dev` was installed. The script `scripts/kconfig/lxdialog/check-lxdialog.sh` checks for ncurses by compiling a small C snippet containing `main() {}`. Since GCC 14, a function without a declared return type (implicit int) is an error rather than a warning, so the test compile fails and the script wrongly concludes that ncurses is missing. We changed it to `int main() { return 0; }` with `sed`, and menuconfig then worked normally.

Remarks

Neither problem came from a wrong configuration; both came from old sources meeting a new compiler, and both were found the same way: by reading the actual compile command in the log instead of only the final error message. The U-Boot part showed clearly what each component of the boot chain is responsible for: the kernel panic in part B was not a kernel bug but a missing link (the rootfs), and once the initramfs was loaded the system went all the way to the shell.